Materia: Programación III

UNIDAD 1 A 6 · DEL OBJETO A LA TRANSACCIÓN

TRABAJO PRÁCTICO

Taller mecánico: órdenes de trabajo, stock y acceso con FastAPI y PostgreSQL

Objetivos

El trabajo integra en un único proyecto lo desarrollado en los capítulos 1 a 6: los fundamentos de FastAPI (capítulo 1), los contratos y el CRUD (capítulo 2), la ejecución asincrónica sin bloqueos (capítulo 3), la persistencia relacional con migraciones (capítulo 4), la autenticación y la autorización (capítulo 5), y los repositorios, el Unit of Work y las transacciones (capítulo 6).

Al finalizar, el proyecto entregado debe demostrar que el alumno puede:

1. Construir una API asincrónica en capas —**Router** → **Service** → **UoW** → **Repository** → **Model**— donde cada capa sabe lo suyo y nada más.
2. Traducir un modelo con asociación, composición, uno a uno y una tabla de catálogo a un esquema relacional versionado con Alembic, incluida una migración que hay que escribir a mano.
3. Autenticar con contraseñas guardadas con bcrypt y un token en una cookie `HttpOnly`, sin frenar el servidor y sin revelar qué cuentas existen.
4. Autorizar consultando los roles **en la base** en cada petición, con vigencia, y distinguir siempre entre 401 y 403.
5. Agrupar en una sola transacción las escrituras de una operación de negocio, de modo que un fallo en el medio no deje datos a medias.
6. Delegar en la base las garantías que la base puede dar: unicidad, claves foráneas, comprobaciones y el sólo agregado.

QUÉ QUEDA AFUERA

No se evalúa la concurrencia (capítulo 7). Dos órdenes que se terminan al mismo tiempo pueden leer las dos el mismo stock, darlo las dos por suficiente y descontar cada una sobre el valor que leyó: la segunda escritura pisa a la primera y el taller entrega más repuestos de los que tenía, sin que el stock llegue a ser negativo —el CHECK lo impediría—. Es una **actualización perdida**: el nivel de aislamiento por defecto la permite, y el bloqueo que la evita es tema de la próxima clase. Tampoco hay colas de tareas con Redis (capítulo 8): el aviso al cliente se resuelve con `BackgroundTasks`, que corre en el mismo proceso. Por eso comparte dos de los tres problemas que la sección 3.9 le señala a una tarea suelta: si el proceso se reinicia, el aviso se pierde, y no hay reintento. Acá se acepta porque el aviso es simulado y perderlo no rompe nada; el capítulo 8 lo resuelve.

Contexto

Un taller mecánico necesita una API para su trabajo diario. La **recepción** da de alta a los clientes y a sus vehículos, y abre una **orden de trabajo** por cada ingreso, con la falla que describe el cliente y los repuestos que se van a usar. Un **mecánico** toma la orden, la repara y la da por terminada: en ese momento se descuenta del stock cada repuesto usado. El **administrador** mantiene el catálogo de repuestos, repone el stock y administra al personal y sus roles. Los **clientes** se registran solos en la web y consultan el estado de sus vehículos.

El dominio está elegido para que cada capítulo tenga un lugar concreto donde aparecer:

Lo que pide el dominio	Por qué	Capítulo
La orden nace con sus ítems; un ítem no existe sin su orden	Composición: los datos de cada ítem viajan dentro del alta de la orden	4
Un cliente puede tener o no un usuario, y a lo sumo uno	Uno a uno opcional: `cliente.usuario_id` UNIQUE y nulo permitido	4
Un vehículo que se vendió se da de baja, pero sus órdenes viejas siguen, y el comprador lo puede registrar con la misma patente	Borrado lógico e índice único parcial	4 y 6
Cuatro roles, que el administrador asigna y revoca con vigencia	La autorización se decide en la base, no en el token	5
El registro crea el usuario, su rol y su ficha de cliente	Tres escrituras que no tienen sentido por separado	6
Terminar una orden descuenta el stock de cada repuesto	Cuatro clases de escritura en una sola transacción	6

Lo que pide el dominio	Por qué	Capítulo
El límite de intentos de login se calcula contando una tabla	El intento tiene que sobrevivir al rollback del login fallido	5 y 6
Los estados de una orden son una lista fija que el sistema necesita para funcionar	Catálogo: una tabla de referencia, apuntada con una clave foránea y cargada por `seed.py`	4
El historial de estados, los movimientos de stock y los cambios de precio no se corrigen	Sólo agregado, impuesto por la base	6

Requisitos técnicos

Componente	Versión mínima	Uso
Python	3.12	Lenguaje
FastAPI	0.111	Framework web; todos los handlers con `async def`
SQLModel	0.0.24	Modelos de tabla
SQLAlchemy	2.0, asincrónico	Motor, sesión y carga anticipada
asyncpg	0.30	Controlador asincrónico de PostgreSQL (se acepta psycopg 3 asincrónico, el del capítulo 4)
PostgreSQL	16	Base de datos
Alembic	1.13, plantilla asincrónica	Migraciones
pydantic-settings	2.0	Configuración leída del entorno
bcrypt	4.0	Hash de las contraseñas
PyJWT	2.8	Token firmado
uvicorn	El que instala `fastapi[standard]`	Servidor ASGI

La configuración se lee con una clase `Settings` de pydantic-settings, desde las variables de entorno o un archivo `.env` que no se versiona. La dirección de la base se arma con las variables sueltas `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `POSTGRES_HOST` y `POSTGRES_PORT`, con un driver asincrónico (`postgresql+asyncpg` o `postgresql+psycopg`). La clave de firma de los tokens también sale del entorno, y ni ella ni la contraseña de la base tienen un valor por defecto en el código: si faltan, la aplicación no arranca. La razón es que la clave de firma es el secreto más importante del sistema: quien la tenga fabrica un token válido para cualquier usuario

(sección 5.12). Y una clave inventada al arrancar tampoco sirve: cambia en cada reinicio, y con dos procesos atendiendo cada uno rechaza los tokens del otro. El repositorio incluye un `env.example` con todas las variables y ningún secreto.

Modelo de dominio

Los nombres de tablas y columnas son obligatorios: los casos de prueba y la corrección los usan. Se pueden agregar columnas, pero no quitar ni renombrar las indicadas.

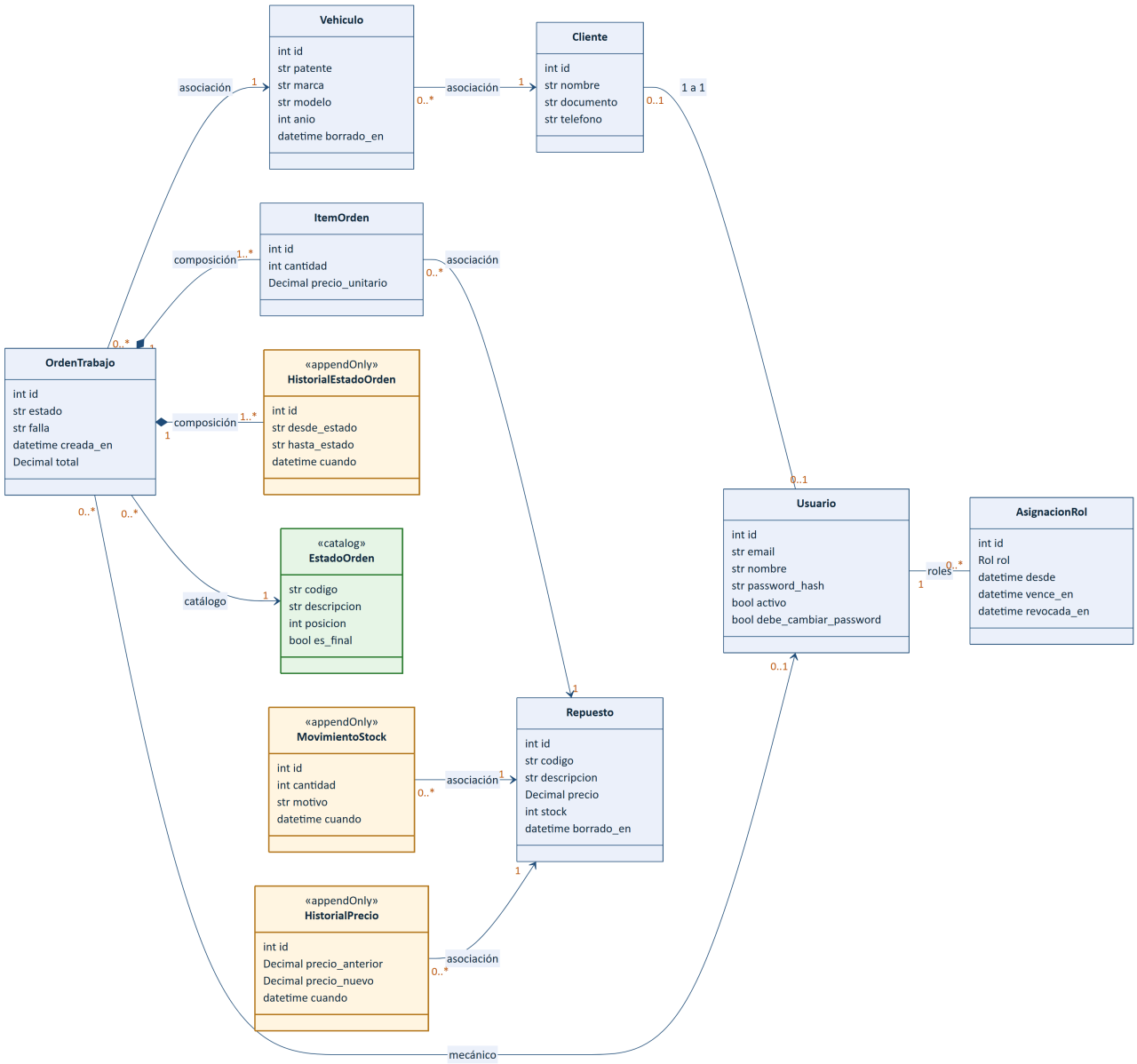


Figura 1 · Diagrama de clases. El rombo lleno es composición y la flecha es asociación. «catalog» (en verde) marca la tabla de referencia que carga `seed.py`, y «appendOnly» (en ámbar), las tablas de sólo agregado que protege el disparador. `IntentoAcceso` y `EventoSeguridad` no aparecen: no se relacionan con ninguna otra tabla.

Tabla	Columnas	Restricciones
`usuario`	`id`, `email`, `nombre`, `password_hash`, `activo`, `debe_cambiar_password`, `password_cambiada_en` (con zona, nulo permitido)	`email` UNIQUE
`asignacion_rol`	`id`, `usuario_id`, `rol`, `desde`, `vence_en` (nulo), `revocada_en` (nulo), `otorgada_por` (nulo)	CHECK de `rol` dentro de ADMIN, RECEPCION, MECANICO, CLIENTE; **índice único parcial** (`usuario_id`, `rol`) `WHERE revocada_en IS NULL`
`intento_acceso`	`id`, `cuando`, `email`, `ip`, `exito`	Índice (`email`, `ip`, `cuando`)
`evento_seguridad`	`id`, `cuando`, `actor` (texto), `accion`	Sin restricciones extra
`cliente`	`id`, `usuario_id` (nulo permitido), `nombre`, `documento`, `telefono`	`usuario_id` UNIQUE; `documento` UNIQUE
`vehiculo`	`id`, `cliente_id`, `patente`, `marca`, `modelo`, `anio`, `borrado_en` (nulo)	**Índice único parcial** (`patente`) `WHERE borrado_en IS NULL`; CHECK `anio BETWEEN 1950 AND 2100`
`repuesto`	`id`, `codigo`, `descripcion`, `precio` (NUMERIC 10,2), `stock`, `borrado_en` (nulo)	**Índice único parcial** (`codigo`) `WHERE borrado_en IS NULL`; CHECK `precio > 0`; CHECK `stock ⩾ 0`
`estado_orden`	`codigo` (clave primaria), `descripcion`, `posicion`, `es_final`	**Catálogo**; `posicion` UNIQUE
`orden_trabajo`	`id`, `vehiculo_id`, `mecanico_id` (nulo), `estado`, `falla`, `creada_en`, `total` (NUMERIC 12,2)	`estado` es clave foránea a `estado_orden.codigo`; CHECK `total ⩾ 0`
`item_orden`	`id`, `orden_id`, `repuesto_id`, `cantidad`, `precio_unitario` (NUMERIC 10,2)	CHECK `cantidad > 0`; CHECK `precio_unitario > 0`; UNIQUE (`orden_id`, `repuesto_id`)

Tabla	Columnas	Restricciones
`historial_estado_orden`	`id`, `orden_id`, `desde_estado` (nulo), `hasta_estado`, `usuario_id`, `cuando`	`desde_estado` y `hasta_estado`, claves foráneas a `estado_orden.codigo`; **sólo agregado**
`movimiento_stock`	`id`, `repuesto_id`, `orden_id` (nulo), `usuario_id`, `cantidad`, `motivo`, `cuando`	CHECK `cantidad <> 0`; **sólo agregado**
`historial_precio_repuesto`	`id`, `repuesto_id`, `usuario_id`, `precio_anterior` (NUMERIC 10,2), `precio_nuevo` (NUMERIC 10,2), `cuando`	CHECK `precio_nuevo > 0`; CHECK `precio_nuevo <> precio_anterior`; **sólo agregado**

Todas las fechas son `timestamptz`. El dinero es `NUMERIC` y en Python `Decimal`, nunca `float` (sección 2.7). Cada clave foránea lleva un índice, porque PostgreSQL no lo crea solo (sección 4.10).

LA PATENTE Y EL CÓDIGO NO SON UNIQUE COMUNES

`vehiculo` y `repuesto` tienen borrado lógico: la fila dada de baja sigue en la tabla. Con un UNIQUE común, el comprador de un auto vendido no podría registrarlo, y la base respondería «ya existe» de un vehículo que la aplicación no muestra en ningún listado. Por eso la unicidad es sólo entre los vigentes: un índice único parcial con `WHERE borrado_en IS NULL`, la misma herramienta que `asignacion_rol` (sección 4.9).

LOS ROLES VIVEN EN SU PROPIA TABLA

`usuario` no tiene una columna `rol`. Los roles de una persona son las filas de `asignacion_rol` que están **vigentes**: no revocadas y sin vencer. Revocar un rol escribe `revocada_en`, y la fila no se borra nunca: así queda el historial de quién tuvo qué rol y hasta cuándo (sección 5.9).

UN CATÁLOGO EN LUGAR DE UN CHECK

Los estados de una orden podrían escribirse en un CHECK con la lista adentro. Con el catálogo `estado_orden`, la lista es un conjunto de filas: la base sigue rechazando un estado que no exista —ahora por la clave foránea— y cada estado lleva datos propios que la API puede mostrar y las consultas pueden cruzar, como `es_final`, que decide qué es una orden abierta (RN-16). La clave primaria es el código y no un `id`, para que `orden_trabajo.estado` se lea igual en `psql` que en la API. La migración crea la tabla **vacía**: sus filas son datos, y las carga `seed.py`.

Lo que el catálogo **no** hace es que un estado nuevo funcione solo. En el código los estados se nombran con un enum, que es el contrato de la API —un estado que no declara da 422— y el vocabulario con que el servicio decide las transiciones. El enum y el catálogo tienen que coincidir, y agregar un estado obliga a tocar los dos y el servicio.

Reglas de negocio

Regla	Enunciado	Garante esperado
RN-01	El correo de un usuario y el documento de un cliente son únicos. La patente de un vehículo y el código de un repuesto son únicos entre los que no están dados de baja.	UNIQUE e índices únicos parciales en la base; 409 con mensaje propio
RN-02	El auto-registro crea el usuario, su rol CLIENTE y su ficha de cliente: o quedan las tres filas, o ninguna. Si el documento ya es de un cliente del taller, el registro da 409 y no se apropia de esa ficha.	Un solo Unit of Work
RN-03	Un usuario tiene a lo sumo una asignación abierta por rol.	Índice único parcial; 409
RN-04	Los permisos se deciden con los roles vigentes en la base, en cada petición. Los roles que trae el token son informativos.	Dependencia que consulta `asignacion_rol`
RN-05	Un administrador no puede revocarse su propio rol ADMIN, y el sistema nunca queda sin administradores vigentes.	Comprobación en el servicio; 409
RN-06	Mientras un usuario tenga el cambio de contraseña pendiente, sólo puede usar las rutas de `/auth` y las de salud.	Dos grupos de rutas; 403
RN-07	Al cambiar la contraseña, las sesiones emitidas antes dejan de valer, aunque sean del mismo segundo que el cambio; la que devuelve el propio cambio, no.	Comparación de `iat` con `password_cambiada_en`, sin truncar; 401

Regla	Enunciado	Garante esperado
RN-08	Después de 5 intentos fallidos con el mismo correo desde la misma dirección en 60 segundos, el login responde 429 sin verificar la contraseña. Un login correcto reinicia la cuenta: los fallos anteriores siguen en la tabla, pero dejan de contar.	Conteo sobre `intento_acceso`; `Retry-After`
RN-09	El login responde lo mismo, y tarda lo mismo, exista o no el correo.	Mismo mensaje; verificación contra un hash ficticio
RN-10	Una orden nace PENDIENTE con al menos un ítem y sin repuestos repetidos. El precio de cada ítem se copia del repuesto, y el total lo calcula el servidor.	DTO; el precio y el total nunca viajan en el cuerpo
RN-11	Las transiciones de una orden son PENDIENTE → EN_REPARACION (la inicia un mecánico, que queda asignado), EN_REPARACION → TERMINADA (la termina el mecánico asignado o un administrador) y PENDIENTE → CANCELADA.	Servicio; 409 si el estado no lo permite
RN-12	Cancelan una orden la recepción, el administrador o **el dueño del vehículo**. El permiso se evalúa antes que el estado.	Servicio; 403 antes que 409
RN-13	Terminar una orden descuenta el stock de cada ítem, deja un movimiento por cada salida y una línea de historial. Si un repuesto no alcanza, no queda ninguna de esas escrituras.	Un solo Unit of Work; 409
RN-14	El stock de un repuesto sólo cambia a través de movimientos: el alta con stock inicial, la reposición y la terminación de órdenes. No se edita por PATCH.	Punto único de escritura de stock
RN-15	`historial_estado_orden`, `movimiento_stock` y `historial_precio_repuesto` son de sólo agregado: nadie puede modificar ni borrar sus filas.	Disparador en la base, instalado por migración
RN-16	Un vehículo o un repuesto dado de baja deja de aparecer y no se puede usar en órdenes nuevas, pero sus datos se conservan. Un vehículo con órdenes abiertas —en un estado que el catálogo no marca como final— no se da de baja.	Borrado lógico; `es_final`; 409
RN-17	Lo propio sólo pide estar autenticado: un cliente ve sus vehículos y las órdenes de sus vehículos, y ninguna otra.	`/mis-*` y comprobación de dueño; 403
RN-18	Ningún error devuelve al cliente texto interno de la base, que le entregaría el mapa del esquema (sección 4.13). Ninguna entrada inválida termina en un 500.	Traducción del `IntegrityError`; validación en el contrato

Regla	Enunciado	Garante esperado
RN-19	Los estados posibles de una orden son las filas del catálogo `estado_orden`. El catálogo se consulta desde la API y no se edita desde ella. Si está vacío, crear una orden responde 409 con un mensaje propio.	Clave foránea; repositorio de sólo lectura
RN-20	Cada cambio del precio de un repuesto deja una línea en `historial_precio_repuesto`, en la misma transacción que el cambio. Un PATCH que no cambia el precio no deja línea.	Un solo Unit of Work; CHECK `precio_nuevo <> precio_anterior`

Estructura obligatoria del proyecto

La organización es por módulo funcional, con las mismas seis capas en todos los módulos:

Ruta	Contenido
`main.py`	Aplicación, lifespan y los dos grupos de rutas
`seed.py`	Deja la base en el estado inicial conocido
`app/core/config.py`	`Settings` con pydantic-settings
`app/core/database.py`	Las funciones que crean el motor y la fábrica de sesiones (las llama el lifespan), y `get_session`
`app/core/security.py`	Hash con bcrypt fuera del bucle, y emisión y verificación del token
`app/core/dependencies.py`	Usuario actual, roles vigentes, control del cambio obligatorio y `requiere_rol`
`app/core/repository.py`	`BaseRepository[T]` y `SoftDeleteRepository[T]`
`app/core/paginacion.py`	El envoltorio de los listados y el cálculo de las páginas
`app/core/unit_of_work.py`	`UnitOfWork` base: confirma o revierte al salir del bloque
`app/modules/<módulo>/models.py`	Modelos de tabla, restricciones y relaciones
`app/modules/<módulo>/schemas.py`	Contratos de entrada y de salida
`app/modules/<módulo>/repository.py`	Los repositorios de cada entidad del módulo
`app/modules/<módulo>/unit_of_work.py`	El Unit of Work del módulo, con los repositorios que sus operaciones necesitan
`app/modules/<módulo>/service.py`	Las reglas de negocio. Recibe el Unit of Work y lo abre

Ruta	Contenido
`app/modules/<módulo>/router.py`	Rutas HTTP: construyen el Unit of Work, llaman al servicio y traducen errores
`migrations/`	Configuración de Alembic y sus versiones
`test/`	Archivos `.http` y `solo_agregado.sql` con los casos de prueba

Módulos mínimos: `health`, `usuarios`, `clientes`, `repuestos` y `ordenes`. `AsignacionRol`, `IntentoAcceso` y `EventoSeguridad` viven en `usuarios`; `Vehiculo` en `clientes`; `MovimientoStock` y `HistorialPrecio` en `repuestos`; `EstadoOrden`, `ItemOrden` e `HistorialEstadoOrden` en `ordenes`. El módulo `health` sólo tiene `router.py`.

Consignas

Ejercicio 1 · Proyecto, configuración y ciclo de vida

8 puntos

Crear el proyecto con la estructura obligatoria. La configuración se lee con `Settings` desde el entorno y la dirección de la base se arma con un `@computed_field`. El motor asincrónico, con el tamaño del pool declarado en la configuración (sección 4.5), y la fábrica de sesiones se crean en el lifespan y se guardan en `app.state`, no al importar un módulo (EA-08, sección 1.9): un recurso construido al importar se crea antes de que exista el bucle de eventos, y el síntoma —anda al levantar el servidor y falla en las pruebas, o al revés— no señala la causa. `get_session` toma la fábrica de `app.state`, entrega una sesión por petición, creada con `expire_on_commit=False` (sección 4.5), y la cierra al terminar. El lifespan calcula además, una vez, el hash ficticio del login, y al apagar cierra el motor con `dispose()`. Se exponen `GET /health/live`, que no toca la base, y `GET /health/ready`, que ejecuta `SELECT 1` y responde 503 si la base no contesta: el proceso puede estar vivo y no estar listo, y reiniciarlo no arregla una base caída.

Ejercicio 2 · Modelos, relaciones y migraciones

14 puntos

Escribir los modelos del modelo de dominio con todas sus restricciones, incluidos los tres índices únicos parciales (`postgresql_where`): el de `asignacion_rol` y los de la patente y el código. El estado de una orden y los del historial son claves foráneas al catálogo `estado_orden`, no un CHECK con la lista escrita adentro (RN-19). Declarar la navegación en los dos extremos de cada relación que se lea. `Usuario.cliente` devuelve un objeto y no una lista. El esquema se crea **exclusivamente** con Alembic, con la plantilla asincrónica: no se usa `create_all`. Se entregan dos migraciones: la primera, generada con `--autogenerate`, con todas las tablas; la segunda, **escrita a mano**, con el disparador de sólo agregado sobre las tres tablas (RN-15), porque la comparación automática no ve funciones ni disparadores. Cada migración se lee antes de aplicarla y lleva un comentario que lo diga. `alembic upgrade head` sobre una base vacía, `alembic downgrade base` y otra vez `upgrade head` deben funcionar. Las migraciones crean el catálogo vacío; sus cuatro filas las carga `seed.py`.

Ejercicio 3 · Contratos y CRUD

10 puntos

Implementar los endpoints de referencia con contratos separados de los modelos de tabla. Los contratos de entrada rechazan los campos que no declaran (`extra="forbid"`), de modo que nadie se registra con un rol ni edita el stock por PATCH. Ninguna respuesta incluye `password_hash`. Los PATCH aplican sólo los campos enviados (`exclude_unset`, sección 2.6). Los códigos son 201 al crear, 204 al dar de baja, 404 si el recurso de la ruta no existe, 409 si choca con lo guardado y 422 si el cuerpo o los parámetros no cumplen el contrato. Los listados devuelven el envoltorio de la sección 2.9, `{items, total, page, size, pages}`: `page` empieza en 1, `size` va de 1 a 100 (por defecto 20), pedir una página más allá de la última da 422, y `total` cuenta con **los mismos filtros** que la página. Con cero resultados hay una página, vacía. La búsqueda de repuestos por descripción usa parámetros ligados.

Ejercicio 4 · Autenticación

16 puntos

Las contraseñas se guardan con bcrypt, con un costo declarado en la configuración y no fijo en el código, porque es un valor que se sube con los años (secciones 5.4 y 5.12), y toda llamada a bcrypt corre fuera del bucle de eventos — `to_thread` — y bajo un semáforo que limita cuántas corren a la vez (sección 5.6). El hash se calcula **antes** de abrir la transacción, porque adentro la conexión quedaría tomada los 300 ms que tarda bcrypt sin usar la base. Todo campo que termina en bcrypt —también la contraseña del login y la actual del cambio— acepta como máximo 72 bytes: bcrypt no mira más allá, y desde la versión 5 rechaza con una excepción lo que pasa de ahí, así que sin el límite en el contrato una contraseña larga daría un 500 en lugar de un 422. Son bytes y no caracteres: una «ñ» ocupa dos. El login sigue este orden (sección 5.5): primero el límite de intentos (RN-08), que cuesta una consulta y, si se superó, responde 429 con `Retry-After` sin calcular ningún hash; después la verificación, contra el hash del usuario o contra un hash ficticio si el correo no existe (RN-09). El token es un JWT firmado con `sub`, `iat`, `exp` y los roles vigentes al emitirlo (sección 5.3). Viaja **sólo** en una cookie `access_token` con `HttpOnly`, `SameSite=Lax`, `Max-Age` igual a la duración del token y `Secure` configurable; el cuerpo del login no lo trae. La dependencia lo lee de `request.cookies`. Existen el registro (RN-02), el cierre de sesión, que borra la cookie, y `GET /auth/yo`, que devuelve el usuario, sus roles vigentes y los que dice el token.

Ejercicio 5 · Autorización

14 puntos

`get_usuario_actual` responde 401 si no hay cookie, si el token es inválido o venció, si el usuario no existe o está inactivo, o si el token es anterior al último cambio de contraseña (RN-07). `requiere_rol` responde 403 y consulta los roles **vigentes en la base** en cada petición (RN-04): revocar un rol le quita el permiso a una sesión ya abierta, sin esperar a que el token venza (sección 5.9). El administrador asigna roles con vencimiento opcional y los revoca sin borrar la fila; reasignar un rol vencido cierra la fila anterior y abre una nueva. El vencimiento llega con zona horaria y en el futuro: una fecha sin zona es ambigua —¿la hora de quién?— y una que ya pasó no choca con nada guardado; las dos son 422, no 409. Se cumple la autoprotección (RN-05). El cambio obligatorio de contraseña se resuelve con **dos grupos de rutas** en `main.py` —uno abierto y uno protegido, que lleva el control montado una sola vez— porque FastAPI suma dependencias y no las resta (sección 5.10). Lo propio sólo pide estar autenticado (RN-17). El historial de seguridad registra las altas, los cambios de rol y de contraseña, y los datos iniciales con el actor `SISTEMA`, que es un texto y no un usuario (sección 5.8).

> **EL `IAT` NO SE TRUNCA A SEGUNDOS**
>
> El token que entrega el cambio de contraseña tiene que valer, y los de las sesiones anteriores no (RN-07). Si `iat` y `password_cambiada_en` se comparan truncados a segundos, una sesión iniciada en el mismo segundo que el cambio, pero antes, es indistinguible de la nueva y sigue valiendo. El estándar de JWT admite fracciones de segundo en `iat`: el token se emite con la hora completa, la comparación se hace sin truncar, y el token nuevo pasa porque se emite después de anotar el cambio (sección 5.10).

Ejercicio 6 · Repositorios y Unit of Work

20 puntos

`BaseRepository[T]` recibe en su constructor la sesión y **la clase del modelo**, porque los genéricos se borran en tiempo de ejecución (sección 6.4.3), y ofrece al menos `get_by_id`, `get_all`, `count`, `add`, `update` y `delete`. `SoftDeleteRepository[T]` hereda de él, filtra los dados de baja y agrega `get_by_id_including_deleted` y `soft_delete`, que no existen en la clase base (sección 6.4.2). `add` y `update` hacen `flush()`, nunca `commit()`: **ningún repositorio confirma** (sección 6.4.4). Cada entidad tiene su repositorio, con métodos que se nombran por lo que traen, y **la precarga se declara adentro del método que la necesita** (sección 6.5). El `UnitOfWork` base es un gestor de contexto asincrónico: confirma al salir del bloque, revierte si se sale con una excepción y **no cierra la sesión**, porque no la creó (secciones 6.6 y 6.7). Cada módulo tiene su Unit of Work, que compone los repositorios que sus operaciones necesitan, todos con la misma sesión. El router construye el Unit of Work con una dependencia y se lo pasa al servicio sin abrirlo; el servicio lo abre con `async with` y nunca lo construye. Las operaciones de RN-02, RN-13 y RN-20 se escriben cada una en un solo bloque. El catálogo tiene un repositorio propio de **sólo lectura** que no hereda de `BaseRepository`: su clave primaria no es `id` y desde la API no se escribe, así que heredar le daría `add`, `update` y `delete` que no debe ofrecer. El stock se escribe en **un único punto** (RN-14), que usan la reposición y la terminación de órdenes.

Ejercicio 7 · Integridad: sólo agregado e intento de acceso

10 puntos

El disparador de la segunda migración rechaza cualquier `UPDATE` o `DELETE` sobre `historial_estado_orden`, `movimiento_stock` y `historial_precio_repuesto`, venga de donde venga (RN-15, sección 6.9). Cada intento de login, correcto o fallido, se anota en `intento_acceso` desde **una sesión y una transacción propias**, que se confirman siempre: es la única escritura del sistema que no comparte la transacción de su petición, porque el fallo que se quiere contar es el mismo que la revierte (sección 6.8.1). El código lo declara en un comentario, junto con su costo: deja filas aunque la petición falle. La sección 6.9 describe un segundo mecanismo independiente para el sólo agregado —que el rol de base con el que se conecta la aplicación no tenga permiso de modificar esas tablas—; no se pide, porque el TP usa un solo rol de base, y según esa misma sección cualquiera de los dos alcanza: tener los dos es defensa en profundidad.

Ejercicio 8 · Lecturas sin N+1 y asincronía — 8 puntos

Toda respuesta que incluya datos de una relación los obtiene con carga anticipada declarada en el repositorio. El detalle de una orden —con su vehículo, el cliente, los ítems, el repuesto de cada ítem y el historial— se resuelve con un número fijo de consultas, que no crece con la cantidad de ítems (sección 4.6). Ninguna lectura depende de la carga perezosa (sección 3.8). Todos los handlers son `async def` y ninguno ejecuta una llamada bloqueante (sección 3.6). Al terminar una orden se agenda con `BackgroundTasks` el aviso al cliente, simulado con `await asyncio.sleep(0.5)` y un mensaje en el registro, que corre **después** de enviar la respuesta y sólo si la operación se confirmó (sección 3.9).

Endpoints de referencia

Grupo	Método	Ruta	Quién	Códigos
Abierto	GET	`/health/live`	Cualquiera	200
Abierto	GET	`/health/ready`	Cualquiera	200, 503
Abierto	POST	`/auth/registro`	Cualquiera	201, 409, 422
Abierto	POST	`/auth/login`	Cualquiera	200, 401, 422, 429
Abierto	POST	`/auth/logout`	Cualquiera	200
Abierto	GET	`/auth/yo`	Autenticado	200, 401
Abierto	POST	`/auth/cambiar-password`	Autenticado	200, 401, 422
Protegido	POST	`/usuarios` (alta de personal, con contraseña provisoria)	ADMIN	201, 409, 422
Protegido	GET	`/usuarios/{id}/roles`	ADMIN	200, 404
Protegido	POST	`/usuarios/{id}/roles`	ADMIN	201, 404, 409, 422
Protegido	DELETE	`/usuarios/{id}/roles/{rol}`	ADMIN	200, 404, 409
Protegido	GET	`/eventos` (`page` , `size`)	ADMIN	200, 422
Protegido	POST	`/clientes`	RECEPCION, ADMIN	201, 409, 422

Grupo	Método	Ruta	Quién	Códigos
Protegido	GET	`/clientes/{id}` (con sus vehículos)	RECEPCION, ADMIN	200, 404
Protegido	POST	`/clientes/{id}/vehiculos`	RECEPCION, ADMIN	201, 404, 409, 422
Protegido	DELETE	`/vehiculos/{id}`	RECEPCION, ADMIN	204, 404, 409
Protegido	GET	`/mis-vehiculos`	Autenticado	200
Protegido	GET	`/repuestos` (`q`, `page`, `size`)	ADMIN, RECEPCION, MECANICO	200, 422
Protegido	GET	`/repuestos/{id}`	ADMIN, RECEPCION, MECANICO	200, 404
Protegido	POST	`/repuestos` (con `stock_inicial`)	ADMIN	201, 409, 422
Protegido	PATCH	`/repuestos/{id}` (descripción y precio)	ADMIN	200, 404, 422
Protegido	DELETE	`/repuestos/{id}`	ADMIN	204, 404
Protegido	POST	`/repuestos/{id}/reponer`	ADMIN	200, 404, 422
Protegido	GET	`/repuestos/{id}/movimientos`	ADMIN	200, 404
Protegido	GET	`/repuestos/{id}/precios` (historial de precios)	ADMIN	200, 404
Protegido	POST	`/ordenes`	RECEPCION, ADMIN	201, 409, 422
Protegido	GET	`/ordenes` (`estado`, `page`, `size`)	ADMIN, RECEPCION, MECANICO	200, 422
Protegido	GET	`/ordenes/{id}`	El personal, o el dueño del vehículo	200, 403, 404
Protegido	POST	`/ordenes/{id}/iniciar`	MECANICO	200, 404, 409
Protegido	POST	`/ordenes/{id}/terminar`	El mecánico asignado, o ADMIN	200, 403, 404, 409

Grupo	Método	Ruta	Quién	Códigos
Protegido	POST	`/ordenes/{id}/cancelar`	RECEPCION, ADMIN o el dueño	200, 403, 404, 409
Protegido	GET	`/estados-orden` (el catálogo, por posición)	Autenticado	200
Protegido	GET	`/mis-ordenes` (`page`, `size`)	Autenticado	200, 422

Toda ruta del grupo protegido responde además 401 sin sesión y 403 con el cambio de contraseña pendiente.

> **LA RUTA DE CANCELAR NO LLEVA `REQUIERE_ROL`**
>
> El dueño de un vehículo puede cancelar su orden y no tiene un rol de personal. Si la ruta exigiera RECEPCION o ADMIN, el dueño recibiría «no tenés permiso» cuando sí lo tiene. La decisión la toma el servicio: primero quién es y qué relación tiene con la orden (403), y recién después si el estado permite cancelarla (409).

Ejemplo del cuerpo de `POST /ordenes`. Viajan el id del vehículo y los datos de cada ítem; no viajan el precio, el total, el estado ni la fecha:

```
{
  "vehiculo_id": 2,
  "falla": "Ruido al frenar",
  "items": [
    { "repuesto_id": 2, "cantidad": 1 },
    { "repuesto_id": 4, "cantidad": 3 }
  ]
}
```

Respuesta esperada, con el precio copiado de cada repuesto, el total calculado y la primera línea del historial:

```
{
  "id": 3,
  "estado": "PENDIENTE",
  "patente": "AC456EF",
  "total": "68000.00",
  "creada_en": "2026-10-23T02:18:54.332849Z",
  "mecanico_id": null,
  "falla": "Ruido al frenar",
  "cliente": "Laura Cliente",
  "items": [
    { "repuesto_id": 2, "codigo": "FRE-010", "descripcion": "Pastillas de freno
delanteras",
      "cantidad": 1, "precio_unitario": "32000.00", "subtotal": "32000.00" },
    { "repuesto_id": 4, "codigo": "ACE-530", "descripcion": "Aceite 5W30 por litro",
      "cantidad": 3, "precio_unitario": "12000.00", "subtotal": "36000.00" }
  ],
  "historial": [
    { "desde_estado": null, "hasta_estado": "PENDIENTE", "usuario_id": 3,
      "cuando": "2026-10-23T02:18:54.366899Z" }
  ]
}
```


La respuesta del login no trae el token: viaja en el encabezado `Set-Cookie`.


```
{ "mensaje": "Sesión iniciada", "debe_cambiar_password": false, "expira_en_segundos":
1800 }
```


Un listado, `GET /repuestos?size=2`, con el envoltorio de la sección 2.9. Con `page=3` respondería 422: «La página 3 no existe: hay 2».


```
{
  "items": [
    { "id": 1, "codigo": "FIL-001", "descripcion": "Filtro de aceite", "precio":
"8500.00", "stock": 10 },
    { "id": 2, "codigo": "FRE-010", "descripcion": "Pastillas de freno delanteras",
"precio": "32000.00", "stock": 4 }
  ],
  "total": 4,
  "page": 1,
  "size": 2,
  "pages": 2
}
```

Datos iniciales

`seed.py` vacía las tablas y carga, como mínimo, estos datos, que los casos de prueba usan. Las contraseñas son de mentira y viven sólo en ese archivo. El catálogo de estados se carga **primero**, porque todo lo que tiene un estado apunta a él, y no es un dato de ejemplo: sin sus filas no se puede crear ninguna orden. Por eso `seed.py` es parte de la puesta en marcha del proyecto y no un extra para probar.

Qué	Datos
Catálogo de estados	PENDIENTE (posición 1), EN_REPARACION (2), TERMINADA (3, final) y CANCELADA (4, final), cada uno con su descripción
Usuarios	`admin@example.com` y `admin2@example.com` (ADMIN), `recepcion@example.com` (RECEPCION), `mecanico@example.com` y `mecanico2@example.com` (MECANICO), `cliente@example.com` (CLIENTE) y `nuevo@example.com` (MECANICO, con el cambio de contraseña obligatorio)
Clientes	Laura, con usuario (`cliente@example.com`) y dos vehículos; Jorge, sin usuario, con un vehículo
Repuestos	Filtro de aceite (stock 10), pastillas de freno (4), bujía (2) y aceite por litro (20), cada stock cargado como un movimiento «Stock inicial»
Órdenes	La 1, PENDIENTE, del primer vehículo de Laura; la 2, EN_REPARACION, del vehículo de Jorge, asignada a `mecanico@example.com` , con **4 bujías** cuando hay 2
Historial de precios	Una línea del filtro de aceite: de 8000 a 8500
Historial de seguridad	Una línea con el actor `SISTEMA` : «Carga de los datos iniciales»

Requisitos no funcionales

Requisito	Detalle
Asincronía	Todos los handlers con `async def` ; bcrypt siempre fuera del bucle
Credenciales	Ninguna contraseña ni clave de firma en el código —tampoco como valor por defecto —, en `alembic.ini` ni en el repositorio
Token	Sólo en la cookie `HttpOnly` ; nunca en el cuerpo ni leído de la cabecera `Authorization`
Esquema	Sólo con Alembic; la aplicación no usa `create_all`
Transacciones	Ningún repositorio confirma; ningún servicio construye su Unit of Work; el Unit of Work no cierra la sesión

Requisito	Detalle
Carga perezosa	Prohibida: toda relación que se lee se precarga en el repositorio
SQL	Nunca armado por concatenación con datos del usuario
Errores	Mensajes propios; nunca el texto de la base ni una traza; ninguna entrada inválida termina en un 500
Unicidad	Garantizada por la base; no se verifica con una consulta previa
Diagnóstico	Las consultas se pueden ver en la consola (`SQL_ECHO=true` o equivalente)

Casos de prueba mínimos

El proyecto incluye archivos `.http` en `test/` que demuestran los casos siguientes, cada uno con la respuesta esperada en un comentario, y un `test/solo_agregado.sql` para el caso C-14. Antes de cada archivo se corre `python seed.py`.

Caso	Qué se demuestra	Resultado esperado
C-01	Un parámetro de ruta que no es entero, con la sesión iniciada	422, sin ejecutar el handler
C-02	Registro; correo y documento repetidos	201; 409 con mensaje propio, y el registro rechazado no deja ninguna fila suelta
C-03	La cookie del login	`Set-Cookie` con `HttpOnly`, `SameSite=Lax` y `Max-Age`; el cuerpo sin token; sin cookie o con `Authorization: Bearer`, 401
C-04	Correo existente con contraseña mala, y correo inexistente	401 con el mismo cuerpo y un tiempo parecido
C-05	Seis intentos fallidos seguidos	El sexto, 429 con `Retry-After` y en pocos milisegundos; los cinco fallos están en `intento_acceso`
C-06	401 contra 403, y lo propio	Sin rol, 403; el cliente ve su orden (200) y no la ajena (403)
C-07	Revocar un rol con la sesión abierta; vencimientos inválidos	La misma cookie, 403 en la petición siguiente; `/auth/yo` muestra la diferencia; la fila revocada sigue. Un vencimiento en el pasado o sin zona horaria, 422
C-08	Autoprotección	El administrador no se revoca su ADMIN: 409

Caso	Qué se demuestra	Resultado esperado
C-09	Cambio obligatorio	Bloqueado (403) salvo `/auth`; después del cambio, la cookie nueva vale y la anterior da 401, sin esperar entre el login y el cambio
C-10	Alta de una orden con dos ítems	201 con los precios copiados, el total y el historial; con un repuesto inexistente, 409 y la orden no queda creada
C-11	Terminar la orden 2, que pide más bujías que las que hay	409; el estado, el stock, los movimientos y el historial quedan exactamente como estaban
C-12	Reponer y terminar la orden 2	200; baja el stock, aparecen los movimientos y la línea de historial; el aviso sale en el registro después de la respuesta
C-13	Cancelar	El dueño cancela su orden pendiente (200); una orden ajena da 403; una terminada, 409
C-14	`UPDATE` y `DELETE` sobre las tres tablas de sólo agregado en `psql`; una orden con un estado que no está en el catálogo	Los seis, rechazados por el disparador; el último, por la clave foránea
C-15	Detalle de una orden con 2 y con 4 ítems	La misma cantidad de consultas en la consola
C-16	Búsqueda con `' OR '1'='1`; paginación con filtro; PATCH parcial; PATCH del stock	Lista vacía con `total` 0; con `q=freno&size=1`, `total` 1 y `pages` 1, y `page=2` da 422; `size=500`, 422; sólo cambia lo enviado; 422
C-17	Catálogo de estados; filtrar órdenes por un estado que el contrato no declara	Los cuatro por posición, con TERMINADA y CANCELADA finales; sin sesión, 401; 422
C-18	Cambiar el precio de un repuesto y después sólo su descripción	Una línea nueva en `/repuestos/{id}/precios` con el precio anterior y el nuevo; el segundo PATCH no agrega ninguna; la recepción recibe 403
C-19	Dar de baja un vehículo sin órdenes y registrarlo con la misma patente para otro cliente	204 y 201: el índice único parcial sólo cuenta los vigentes; una segunda patente vigente igual, 409

VARIAS SESIONES A LA VEZ

Con la extensión «REST Client», cada login se nombra con `# @name` y los pedidos de esa sesión llevan `# @no-cookie-jar` y el encabezado `Cookie: {{loginAdmin.response.headers.Set-Cookie}}`. Así un mismo archivo puede tener la sesión del administrador y la de la recepción al mismo tiempo, que es lo que necesita el caso C-07.

Criterios de evaluación

Criterio	Peso	Qué se mira
Proyecto, configuración y ciclo de vida	8 %	Estructura, `Settings` sin secretos por defecto, motor y fábrica creados en el lifespan, `get_session`, salud
Modelos, relaciones y migraciones	14 %	Restricciones, índices únicos parciales, catálogo con clave foránea, navegación, dos migraciones (una a mano), upgrade y downgrade
Contratos y CRUD	10 %	Contratos separados, `extra="forbid"`, códigos, ninguna entrada inválida en 500, PATCH parcial, envoltorio y paginación de la sección 2.9
Autenticación	16 %	bcrypt fuera del bucle y con semáforo, orden del login, límite, hash ficticio, cookie `HttpOnly`
Autorización	14 %	401 y 403, roles vigentes en la base, autoprotección, dos grupos de rutas, `iat`, lo propio
Repositorios y Unit of Work	20 %	Dos jerarquías, clase del modelo, `flush` y no `commit`, precarga en el repositorio, quién crea, abre, confirma y cierra, operaciones de varias escrituras
Integridad	10 %	Disparador de sólo agregado en las tres tablas; historial de precios en la misma transacción; intento de acceso en su propia sesión
Lecturas sin N+1 y asincronía	8 %	Consultas fijas en el detalle; ninguna llamada bloqueante; aviso en segundo plano

Un proyecto que no levanta, que no crea su esquema con `alembic upgrade head` sobre una base vacía o que contiene credenciales reales en el repositorio no se corrige hasta que se subsane.

Formato de entrega

- Repositorio con el código fuente completo, `requirements.txt`, `env.example`, `alembic.ini`, la carpeta `migrations/` y la carpeta `test/`. El archivo `.env` no se sube.
- `README.md` con los pasos para levantar el proyecto desde cero: entorno virtual, dependencias, `.env` (con cómo generar la clave de firma), `alembic upgrade head`, `python seed.py` y arranque del servidor.
- Evidencia del caso C-15: una captura o un fragmento de la consola con las consultas del detalle de una orden con 2 ítems y de otra con 4.
- Evidencia del caso C-14: una captura de `psql` con los seis rechazos del disparador y el de la clave foránea.